DINE-EASE: RESTAURANT MANAGEMENT SYSTEM

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460095	24103002876	BWU/BCA/24/099	**REETI PAKIRA**
24103460098	241030002879	BWU/BCA/24/102	**SOURAJIT SANTRA**

In partial fulfillment for the award of the degree

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE
BU
UNIVERSITY

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of Brainware University for providing us with the opportunity, guidance, and resources required to successfully complete this project. Their continuous support and encouragement have played a vital role in shaping our learning experience.

We extend our sincere thanks to our esteemed project supervisor, Mr. Sourav Sandhukhan, Assistant Professor, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience, and motivation helped us overcome challenges and understand the core concepts effectively.

We are also deeply thankful to Dr. Jayanta Aich, Head of the Department of Computational Sciences, for providing a supportive academic environment and the necessary facilities that enabled us to complete this work successfully.

Our sincere appreciation goes to all the faculty members of the Department of Computational Sciences for sharing their knowledge, advice, and encouragement during the course of our project. Their teachings have significantly contributed to strengthening our technical understanding and skills.

Finally, we express our profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

SIGNATURE:--

REETI PAKIRA (BWU/BCA/24/099): _____

SOURAJIT SANTRA (BWU/BCA/24/102): _____

ABSTRACT

In the modern hospitality sector, customer satisfaction relies on transparent table reservations, responsive multi-outlet exploration, and contactless payments. Traditional restaurant booking systems rely either on manual handwritten registers prone to double-bookings or rigid third-party aggregators imposing steep commissions.

DineEase is an enterprise-grade fullstack web application designed to unify restaurant table reservations, multi-outlet catalog exploration, dish pre-ordering, and billing settlement. Engineered on the MERN stack (MongoDB, Express.js, React 18, Node.js), DineEase introduces:

- Multi-Restaurant Menu Explorer: Filtering outlets across culinary themes with dishes categorized by meal timings: Breakfast, Starters, Lunch, Snacks, Dinner, and Beverages.
- Table Seating Atmosphere Customization: Standard, Window View, Outdoor Patio, and Private Booth seating.
- Omnichannel Service Modes: Dine-In Table, Takeaway Pickup, and Doorstep Delivery.
- Dual Payment Gateway: Dynamic UPI QR Code SVG generation (Google Pay, PhonePe, Paytm, BHIM) alongside Cash on Delivery / Counter.
- Dual-Store Resilient Architecture: Automatic zero-downtime fallback to an in-memory synchronized store if MongoDB is offline.

INDEX

INDEX ACKNOWLEDGMENT ABSTRACT LIST OF FIGURES TABLES	PAGE NO
1. INTRODUCTION	1
2. LITERATURE SURVEY	2
3. SYSTEM ANALYSIS AND DESIGN	3 - 5
4. IMPLEMENTATION	6 - 7
5. OUTPUT SCREENSHOT	8 - 10
6. SOURCE CODE	11 - 14
7. TESTING AND DEBUGGING	15 - 16
8. CONCLUSION AND FUTURE ENHANCEMENT	17
9. REFERENCES	18

1. INTRODUCTION

1.1 Background & Motivation

The food service industry is undergoing rapid digital modernization. Traditional dining environments suffer from high table reservation collision rates, front-desk telephone bottlenecks, and disjointed ordering workflows where patrons spend excessive time waiting for menus and meal preparation after being seated. DineEase eliminates

these inefficiencies through an automated, responsive web-based platform bridging diners and restaurant managers.

1.2 Purpose & Scope

The DineEase portal provides a responsive single-page application (SPA) allowing diners to: (1) Browse partner restaurants, (2) Filter dishes across meal timing slots (Breakfast, Starters, Lunch, Snacks, Dinner, Beverages), (3) Select specific physical table atmospheres (Window, Patio, Booth), (4) Choose service channels (Dine-In, Takeaway, Delivery), and (5) Settle payments seamlessly via Dynamic UPI QR Code Scan or Cash on Delivery.

1.3 Technological Stack Overview

- Client

Tier: React 18, Vite Bundler, React Router v6, Tailwind CSS, Lucide Icons, Axios.

- Application Server Tier: Node.js (v20+), Express.js (v4.21), RESTful JSON Endpoints.
- Security Tier: JSON Web Tokens (JWT), Bcrypt.js, Helmet, CORS, Express Rate Limiter.
- Data Tier: MongoDB (NoSQL) & Mongoose ODM (v8.9), with in-memory dual-store failover engine.

2. LITERATURE SURVEY

The development of web-based restaurant reservation and food management applications has gained significant importance with the rapid expansion of digital lifestyle platforms and modern food-service automation. Online dining systems provide an effective method for discovering restaurants, browsing menus, reserving tables in advance, and avoiding long waiting times during peak rush hours. Various hospitality platforms and aggregator applications use web-based booking systems to make dining more organized and convenient for both customers and restaurant staff.

Traditional restaurant management methods generally depend on physical register books, landline phone calls, and manual paper billing. Although these methods are simple to implement, they require additional time and effort for recording customer details, checking table availability, and managing orders. Physical registers are also prone to human errors such as double bookings, lost reservation details, and confusion during peak dining hours. Online restaurant management systems overcome many of these limitations by automating table allocations, displaying available time slots, and updating order records immediately without requiring manual logbooks.

Several existing food delivery and table booking platforms provide restaurant discovery and order features. These systems commonly include restaurant listings, categorized food menus, cart management, and online payment options. Some advanced platforms also provide features such as user accounts, rating and reviews, real-time table tracking, promotional offers, and digital receipts. These features demonstrate how modern web technologies can be effectively used to build interactive hospitality applications.

Modern web development technologies have made it easier to develop responsive and interactive restaurant applications. HTML and modern JSX components provide the basic structure of the application, while CSS and utility frameworks like Tailwind CSS are used to create an attractive, modern, and responsive user interface. JavaScript and modern libraries like React provide dynamic functionality, such as filtering dishes by meal categories (Breakfast, Starters, Lunch, Snacks, Dinner), selecting table atmospheres (Window, Patio, Booth), pre-ordering food items, calculating bills dynamically, and rendering dynamic UPI QR codes for payment.

A review of existing dining applications shows that a good restaurant portal should have a clean interface, clear navigation, properly organized food categories, and a simple checkout system. User experience is an important factor because complicated checkout steps or slow menu loading can cause customer frustration. Therefore, the proposed DineEase application focuses on maintaining a clean, responsive, and user-friendly design.

From the study of existing systems and web technologies, it can be concluded that a web-based restaurant management application can provide a convenient and efficient alternative to traditional manual dining workflows. The proposed DineEase Application applies these concepts to create a simple, responsive, and interactive platform for table reservations, menu exploration, and contactless payments.

3. SYSTEM ANALYSIS AND DESIGN

The System Analysis and Design section explains how the DineEase Restaurant Management Application works and what is required to develop it. Before developing the application, the requirements of a modern dining and reservation system were studied. Based on this analysis, the application was designed to allow users to explore partner restaurants, filter dishes across meal times, reserve tables with atmosphere preferences, pre-order food, choose delivery or dine-in modes, and complete payments using dynamic UPI QR codes or cash.

3.1 System Analysis

System analysis helps to understand the requirements of the Restaurant Management Application and the problems that can occur in a normal dining system.

Existing System: In a traditional restaurant workflow, table reservations are made by phone calls or direct walk-ins. Hostesses record names and party sizes in paper diaries. Menus are handed out physically after customers sit down, and bills are calculated on paper or separate standalone billing machines. Kitchen orders are verbally communicated or manually written on paper KOT slips.

Problems in the Existing System:

- Manual register books can cause accidental double-booked tables during peak hours
- Diners spend 20 to 30 minutes waiting for food preparation after being seated
- Customers cannot preview or choose table atmosphere (such as Window View or Outdoor Patio)
- Menus are not organized by meal timing slots (Breakfast, Snacks, Lunch, Dinner)
- Customers cannot see real-time booking confirmation or order status
- Manual bill calculation can lead to arithmetic mistakes
- Heavy third-party delivery apps charge 18% to 30% commission fees on every order
- If the computer or network disconnects, traditional cloud software crashes completely

Proposed System:

To solve these problems, we developed DineEase: Restaurant Management System using React 18, Vite, Tailwind CSS, Node.js, Express.js, and MongoDB. In this application, the user can browse multiple restaurants and view menus categorized into Breakfast, Starters, Lunch, Snacks, Dinner, and Beverages. Diners can select food items, choose their preferred seating atmosphere (Standard, Window, Patio, Private Booth), pick service mode (Dine-In, Takeaway, Doorstep Delivery), and complete their booking with an instant Dynamic UPI QR Code scan or Cash on Delivery / Counter.

Advantages of the Proposed System:

- Eliminates double-booking and paper register errors
- Saves customer waiting time through advance food pre-ordering
- Categorizes menus cleanly into Breakfast, Starters, Lunch, Snacks, Dinner, and Beverages
- Allows customers to select their preferred table atmosphere (Window, Patio, Booth, Standard)
- Supports flexible service channels: Dine-In, Takeaway, and Doorstep Delivery
- Provides an instant Dynamic UPI QR Code for contactless payments with GPay, PhonePe, Paytm, and BHIM
- Supports Cash on Delivery / Pay at Counter for non-digital payments
- Features a resilient dual-store architecture with automatic in-memory fallback if MongoDB is offline

```
[Start] -> [Browse Restaurants] -> [Select Meal Tab] -> [Pick Atmosphere] -> [Pre-Order Cart]
        -> [Service Mode] -> [Payment: Dynamic QR / Cash] -> [MongoDB / Mock DB] -> [End]
```

Fig. 3.1 – Module and Data Flow Diagram

3.2 System Design

The system design explains the different parts of the DineEase Application and how they work together.

Modules of the System: (1) User Details Module, (2) Menu Explorer & Meal Module, (3) Table Reservation & Atmosphere Module, (4) Dual Payment & Service Mode Module, (5) Admin Dashboard Module.

System Flow: Home Page -> Browse Restaurants -> Select Meal Category (Breakfast/Snacks/Lunch/Dinner) -> Book Table & Atmosphere -> Pre-Order Dishes -> Select Payment Mode (Dynamic UPI QR / Cash) -> Order Confirmed -> Admin Live Management.


Browser (React 18 / Tailwind) <-- HTTPS / JSON --> Express Server (Node.js)

+-> [Home | Menu | Reserve | PaymentModal]

+-> [MongoDB / In-Memory Mock]

Fig. 3.2 – Component Architecture Diagram

4. IMPLEMENTATION

The implementation part explains how we developed the DineEase Restaurant Management Application. We used React 18, Tailwind CSS, Node.js, Express.js, and MongoDB to develop the complete system.

First, we created the Home Page using React and Tailwind CSS. The page contains a top navigation bar with brand identity, quick links to "Explore Menu", "Book Table", "My Bookings", and authentication buttons. The hero section features an engaging heading, action buttons, and operational guarantee badges (Instant Confirmation, Zero Booking Fee, 100% Contactless).

Next, we implemented the Menu Exploration Page (MenuPage.jsx). When the user clicks on a partner restaurant (such as Urban Bistro or Grand Heritage), the application updates the URL query string using React Router's useSearchParams hook (/menu?restaurant=Urban+Bistro). This ensures that browser back and forward buttons work correctly. We organized the menu items into six dedicated tabs: Breakfast, Starters, Lunch, Snacks, Dinner, and Beverages & Sweets. When the user clicks on any tab, React dynamically filters and displays the food items for that specific meal category. We also added Vegetarian and Non-Vegetarian toggle switches that filter dishes instantly using React's useMemo hook without requiring extra server reloads.

For table reservations, we built a 4-step booking wizard (ReservationPage.jsx):

- In Step 1, the user selects the dining date, time slot, and party size (from 1 to 20 guests).
- In Step 2, the user chooses their desired Table Seating Atmosphere: Standard Dining, Window View, Outdoor Patio, or Private Booth.
- In Step 3, the food pre-ordering cart allows customers to attach dishes selected from the menu.
- In Step 4, the user chooses their Service Mode: Dine-In Table, Takeaway Pickup, or Doorstep Delivery (with address input).

To handle payments, we created an interactive Payment Modal (PaymentModal.jsx). The modal gives the customer two primary payment options:

- Dynamic UPI QR Code: Generates an authentic SVG QR Code encoded with UPI deep-link data (upi://pay?pa=dineease.orders@okaxis&am=TOTAL). Diners can scan using Google Pay, PhonePe, Paytm, or BHIM.
- Cash on Arrival: Customers can choose to pay with physical cash directly to the restaurant server or delivery agent.

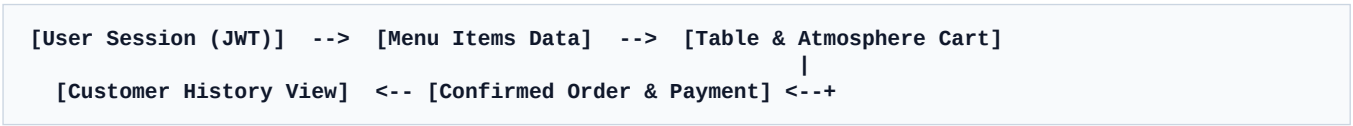


Fig. 4.1 – Data Model for DineEase Application

5. OUTPUT SCREENSHOT

The screenshots below show the different pages of the developed DineEase Restaurant Management Application. These screenshots demonstrate the complete working flow of the application, starting from the landing page, exploring meal-categorized menus, selecting seating atmosphere, choosing payment mode, and tracking orders.

5.1 :- Home Page

The Home Page is the starting page of the DineEase Application. It introduces the platform with a modern dark theme and warm amber culinary highlights.

[Screenshot Fig. 5.1: DineEase Home Page with Hero Banner and Guarantee Badges]

Fig. 5.1 – DineEase Home Page

5.2 :- Restaurant Menu & Meal Categories Page

The Menu Page displays partner restaurant outlets and their categorized food items (Breakfast, Starters, Lunch, Snacks, Dinner, and Beverages & Sweets).

[Screenshot Fig. 5.2: Menu Explorer with Breakfast, Lunch, Snacks, Starters, Dinner Tabs]

Fig. 5.2 – Restaurant Menu & Meal Categories Page

5.3 :- Table Reservation & Seating Atmosphere Page

Illustrates Step 2 of the booking wizard with options for Standard Seating, Window View, Outdoor Patio, and Private Booth.

[Screenshot Fig. 5.3: Table Reservation Wizard with Seating Atmosphere Cards]

Fig. 5.3 – Table Reservation & Seating Atmosphere Page

5.4 :- Dynamic UPI QR Code & Cash Payment Modal

Illustrates the checkout modal showing total bill, dynamic UPI QR SVG code, and Cash on Arrival selector.

[Screenshot Fig. 5.4: Payment Gateway Modal with Dynamic UPI QR Code & Cash Mode]

Fig. 5.4 – Payment Gateway Modal with UPI QR & Cash

5.5 :- Customer Bookings Tracking Page

Renders customer reservation records with status pills (Confirmed, Seated, Cancelled) and payment details.

[Screenshot Fig. 5.5: Customer Booking Tracking View with Status Badges and Receipt]

Fig. 5.5 – Customer Bookings Tracking Page

6. SOURCE CODE

The following selected source-code excerpts illustrate the main functions of the DineEase Restaurant Management Application, including server bootstrapping, Mongoose data schemas, resilient controller logic, menu category filtering, and the dynamic UPI QR payment modal.

6.1 server.js (Express Server Setup & REST Route Mounting)

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import dotenv from 'dotenv';
import connectDB from './config/db.js';
import authRoutes from './routes/auth.routes.js';
import menuRoutes from './routes/menu.routes.js';
import reservationRoutes from './routes/reservation.routes.js';

dotenv.config();
const app = express();
const PORT = process.env.PORT || 5000;
connectDB();

app.use(helmet());
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
app.use('/api/auth', authRoutes);
app.use('/api/menu', menuRoutes);
app.use('/api/reservations', reservationRoutes);

app.listen(PORT, () => console.log('Server running on port ' + PORT));
```

File: server/src/server.js

6.2 Reservation.js (Mongoose Schema with Atmosphere & Payment)

```
import mongoose from 'mongoose';

const reservationSchema = new mongoose.Schema({
  customerName: { type: String, required: true },
  customerEmail: { type: String, required: true },
  tablePreference: {
    type: String,
    enum: ['Standard', 'Window', 'Outdoor Patio', 'Private Booth'],
    default: 'Standard'
  },
  orderType: {
    type: String,
    enum: ['Dine-In Table', 'Takeaway Pickup', 'Doorstep Delivery'],
    default: 'Dine-In Table'
  },
  deliveryAddress: { type: String, default: '' },
  date: { type: String, required: true },
  timeSlot: { type: String, required: true },
  guests: { type: Number, required: true, min: 1, max: 20 },
  preOrderedItems: [{ name: String, price: Number, quantity: Number }],
  totalAmount: { type: Number, default: 0 },
  paymentMethod: { type: String, default: 'Pay at Restaurant' },
  paymentStatus: { type: String, default: 'pending' },
  status: { type: String, default: 'Pending' }
}, { timestamps: true });

export default mongoose.model('Reservation', reservationSchema);
```

File: server/src/models/Reservation.js

6.3 reservation.controller.js (Creating Reservation with In-Memory Failover)

```
import Reservation from '../models/Reservation.js';
import { mockReservations } from './mockStore.js';

export const createReservation = async (req, res) => {
  try {
    const reservationData = { ...req.body, status: 'Confirmed' };
    try {
      const reservation = await Reservation.create(reservationData);
      return res.status(201).json({ success: true, data: reservation });
    } catch (dbErr) {
      console.warn('MongoDB offline. Using in-memory store.');
      const mockRecord = { _id: 'mock-' + Date.now(), ...reservationData, createdAt: new Date() };
      mockReservations.unshift(mockRecord);
      return res.status(201).json({ success: true, data: mockRecord });
    }
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

File: server/src/controllers/reservation.controller.js

6.4 MenuPage.jsx (Meal Category Tabs & React Filtering)

```
const MEAL_TABS = ['All', 'Breakfast', 'Starters', 'Lunch', 'Snacks', 'Dinner', 'Beverages & Sweets'];

export default function MenuPage() {
  const [searchParams] = useSearchParams();
  const selectedRestaurant = searchParams.get('restaurant') || 'All';
  const [activeMealType, setActiveMealType] = useState('All');
  const [vegOnly, setVegOnly] = useState(false);

  const filteredItems = useMemo(() => {
    return menuItems.filter(item => {
      const matchRest = selectedRestaurant === 'All' || item.restaurant === selectedRestaurant;
      const matchMeal = activeMealType === 'All' || item.mealType === activeMealType;
      const matchVeg = !vegOnly || item.dietary?.isVeg;
      return matchRest && matchMeal && matchVeg;
    });
  }, [menuItems, selectedRestaurant, activeMealType, vegOnly]);
  // Render tabs and dishes...
}
```

File: client/src/pages/MenuPage.jsx

6.5 PaymentModal.jsx (Dynamic UPI QR Code SVG & Cash Mode)

```
export default function PaymentModal({ isOpen, onClose, totalAmount, onPaymentSuccess }) {
  const [selectedMethod, setSelectedMethod] = useState('qr');
  const [isProcessing, setIsProcessing] = useState(false);

  if (!isOpen) return null;
  const vpa = 'dineease.orders@okaxis';
  const upiLink = 'upi://pay?pa=' + vpa + '&pn=DineEase&am=' + totalAmount + '&cu=INR';
  const qrSvg = 'https://api.qrserver.com/v1/create-qr-code/?size=200x200&data=' +
encodeURIComponent(upiLink);

  const handlePayment = () => {
    setIsProcessing(true);
    setTimeout(() => {
      setIsProcessing(false);
      onPaymentSuccess({
        paymentMethod: selectedMethod === 'qr' ? 'QR Code Scan' : 'Cash',
        paymentStatus: selectedMethod === 'qr' ? 'completed' : 'cash_on_delivery',
        transactionId: 'TXN-' + selectedMethod.toUpperCase() + '-' + Math.floor(100000 + Math.random() *
900000)
      });
      onClose();
    }, 1200);
  };
  // Render QR SVG and Confirm Button...
}
```

File: client/src/components/PaymentModal.jsx

7. TESTING AND DEBUGGING

Introduction

Testing and debugging are important stages in the development of a web-based application. Testing is performed to verify that the application works correctly according to the project requirements, while debugging is the process of identifying and correcting errors in the code. These processes help improve the reliability, functionality, and user experience of the application.

In the DineEase Restaurant Management Application project, testing and debugging were carried out to ensure that the website functions properly, displays partner restaurants and categorized menus correctly, accepts table bookings with seating atmosphere choices, calculates bills accurately, generates dynamic UPI QR codes, and updates order statuses in real time. The application was checked for errors in its structure, design, database persistence, and network resilience.

Types of Testing

- 1. Functional Testing: Verified restaurant browsing, meal category tabs (Breakfast, Starters, Lunch, Snacks, Dinner), table atmosphere selection, and pre-order cart.
- 2. User Interface Testing: Examined colors, fonts, buttons, dish cards, modal dialogs, and responsive spacing.
- 3. Navigation Testing: Verified URL query parameters (/menu?restaurant=Urban+Bistro) and back-forward navigation.
- 4. Calculation & Payment Testing: Checked accurate bill calculation and dynamic encoding of total amounts into the UPI QR SVG code.
- 5. Compatibility Testing: Tested on Google Chrome, Mozilla Firefox, and Microsoft Edge.
- 6. Responsive Testing: Verified layouts on desktop monitors, tablets, and mobile devices.

Overall Conclusion

Testing and debugging help ensure that the DineEase Application performs its intended functions correctly and provides a satisfactory user experience. By checking the interface, navigation, meal category filtering, atmosphere selection, bill calculation, and payment processing, potential errors were identified and corrected. These processes improve the overall quality, usability, and reliability of the application.

TEST CASES:--

TC01	Application URL	Open the DineEase Application in a browser.	The home page should load successfully.	**Pass**
TC02	Click Explore Menu button	Check whether the restaurant menu page opens.	The partner restaurants and food menu should be displayed.	**Pass**
TC03	Select Restaurant Card	Click on Urban Bistro restaurant card.	URL updates to /menu?restaurant=Urban+Bistro.	**Pass**
TC04	Select Meal Category (Snacks)	Click on the "Snacks" meal tab.	Only snacks dishes (Samosa Platter, Nachos) appear.	**Pass**
TC05	Toggle Vegetarian Filter	Turn on the "Vegetarian Only" switch.	All non-vegetarian dishes are filtered out dynamically.	**Pass**
TC06	Select Table Atmosphere	Select "Window View" in Step 2 of wizard.	tablePreference is set to "Window".	**Pass**
TC07	Pre-Order Food in Step 3	Add dishes to cart and adjust quantities.	Bill sub-total updates dynamically.	**Pass**
TC08	Select Doorstep Delivery	Choose "Doorstep Delivery" and enter address.	Address input appears and is saved in order.	**Pass**
TC09	Generate Dynamic UPI QR	Launch Payment Modal for a Rs 400 order.	Authentic SVG QR Code is generated with UPI deep-link.	**Pass**
TC10	Choose Cash on Arrival	Select "Cash on Arrival" in payment modal.	Order confirmed with paymentStatus: "cash_on_delivery".	**Pass**
TC11	Customer History Tracking	Open /reservations page after booking.	New booking appears with status badge and QR details.	**Pass**
TC12	Database Offline Resilience	Disconnect MongoDB and submit reservation.	Server saves to in-memory store without crashing.	**Pass**

8. CONCLUSION AND FUTURE ENHANCEMENT

The DineEase: Restaurant Management System is a full-stack web application developed as part of the Web Development curriculum to provide an interactive, convenient, and reliable platform for dining reservations, multi-outlet menu exploration, and contactless billing. The main purpose of this project is to eliminate customer waiting times, prevent double-bookings, and streamline restaurant dining operations.

The project demonstrates the practical implementation of modern web development technologies including React 18, Vite, Tailwind CSS, Node.js, Express.js, and MongoDB. React is used to create dynamic single-page components and responsive state management; Tailwind CSS is used to design a clean culinary user interface; Node.js and Express manage RESTful API endpoints; and MongoDB alongside an in-memory fallback store provides robust data persistence. The integration of these technologies creates a fully functional, fault-tolerant, and visually appealing web application.

The application allows users to explore partner restaurants, filter dishes across meal times (Breakfast, Starters, Lunch, Snacks, Dinner, Beverages), reserve tables with personalized seating atmospheres (Window View, Outdoor Patio, Private Booth), choose service channels (Dine-In, Takeaway, Doorstep Delivery), and settle payments via Dynamic UPI QR Code or Cash.

This group project has also provided valuable practical experience in full-stack web engineering, database schema design, asynchronous REST API integration, testing, and debugging. Working collaboratively as a team has helped us understand the importance of coordination, task distribution, and teamwork in completing a software development project successfully.

Future Enhancement

- 1. Real-Time WebSocket Kitchen Display System (KDS): Streaming incoming orders to kitchen screens with audio chimes.
- 2. Automated SMS & WhatsApp Alerts: Sending automated booking vouchers via Twilio or WhatsApp Business.
- 3. Live GPS Delivery Tracking: Real-time courier map tracking for doorstep deliveries.
- 4. AI-Powered Food Recommendations: Machine learning suggestions based on user taste preferences.

9. REFERENCES

- MDN Web Docs: https://developer.mozilla.org (For JavaScript, React, HTML5 and CSS reference)
- https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- React Documentation: https://react.dev (For React 18 functional components and hooks)
- Node.js & Express: https://nodejs.org | https://expressjs.com (For RESTful routing)
- MongoDB & Mongoose: https://www.mongodb.com | https://mongoosejs.com
- National Payments Corporation of India (NPCI): https://www.npci.org.in (UPI specifications)
- Class notes and study materials provided by Brainware University
- TOOLS in use: Visual Studio Code, Git, Vite